



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO 3 – ASSESMENT TOOL 3

NAME: P.V.N. ANISH BABU

REG NO: 192472329

COURSE CODE: CSA0924

COURSE NAME: JAVA PROGRAMMING

SLOT: D

Analyse why Thread.stop(), suspend() and resume() were deprecated. Design a cooperative shutdown mechanism using a volatile flag and interruption for a long-running report generator

AIM

To analyze why Thread.stop(), Thread.suspend(), and Thread.resume() were deprecated in Java and to design a cooperative shutdown mechanism using a volatile flag and thread interruption for a long-running report generator.

Design of a Long-Running Report Generator

ALGORITHM

1. Create a report generator thread.
2. Use a volatile boolean flag named running.
3. Continuously generate report sections while running is true.
4. Use Thread.sleep() to simulate processing.
5. When shutdown is required:
 - Set running = false.
 - Call interrupt().
6. Exit the loop gracefully.

PSEUDO CODE

START

Create ReportGenerator thread

Set running = true

WHILE running is true AND thread is not interrupted

 Display "Generating report section..."

 Sleep for 2 seconds

 IF interruption occurs THEN

 Display "Interrupt received"

 Set interrupted status

 END IF

END WHILE

Display "Report generator stopped safely"

Main thread waits for 7 seconds

Set running = false

Interrupt the ReportGenerator thread

Wait until the thread finishes

Display "Shutdown complete"

STOP

JAVA PROGRAM

```
import threading
```

```
import time
```

```
class ReportGenerator:
```

```
    def __init__(self):
```

```
        self._running = True
```

```
        self._thread = threading.Thread(target=self._run)
```

```
    def shutdown(self):
```

```
        print("\nShutdown signal received.")
```

```
        self._running = False
```

```
        # In Python, we can't directly interrupt a thread like in Java's InterruptedException.
```

```
        # Instead, we rely on the thread checking the _running flag.
```

```
        # For blocking operations, you might need more advanced techniques like event objects or
        timeouts.
```

```
    def _run(self):
```

```
        while self._running:
```

```
            try:
```

```
                print("Generating report section...")
```

```
                time.sleep(2) # Simulate work
```

```
            except Exception as e:
```

```
                print(f"An unexpected error occurred: {e}")
```

```
                break
```

```
print("Report generator stopped safely.")
```

```
def start(self):
```

```
    self._thread.start()
```

```
def join(self):
```

```
    self._thread.join()
```

```
# Main execution
```

```
if __name__ == "__main__":
```

```
    generator = ReportGenerator()
```

```
    generator.start()
```

```
# Let the generator run for a while
```

```
time.sleep(7)
```

```
# Signal for shutdown
```

```
generator.shutdown()
```

```
# Wait for the thread to finish
```

```
generator.join()
```

```
print("Shutdown complete.")
```

OUTPUT

Generating report section...

Generating report section...

Generating report section...

Generating report section...

Shutdown signal received.

Report generator stopped safely.

Shutdown complete.

Advantages of Cooperative Shutdown

- Prevents data corruption.
- Avoids deadlocks.
- Releases resources properly.
- Allows cleanup operations.
- Ensures graceful thread termination.

CONCLUSION

`Thread.stop()`, `Thread.suspend()`, and `Thread.resume()` were deprecated because they can cause deadlocks, inconsistent object states, and unpredictable behavior. A cooperative shutdown mechanism using a volatile flag and thread interruption provides a safe and efficient way to terminate long-running tasks such as a report generator.